Name:

Points: 6 /26 VG

Databasutveckling och lagring Omtenta Normalisering

2025-05-26

Instructions

1. **No computers, no phones, no notes**
2. **No talking with other people in the room**
3. In case point 1 or 2 are broken, you get automatically **0 points** on the test.
4. Write your name on top of the page (do not fill in the *Points* box!)
5. You will be presented with a database schema that isn't normalized. There are **6 normalization problems** in the given schema.
6. For each problem you need to write shortly (2 short sentences) about:
   - what the problem is
   - in which way it breaks normalization
   - how to fix it

   Example:

   > *The column X in the table Y breaks normalization by ... . If we ..., we can solve the issue.*

7. Once you're done:
   - Double-check your answers
   - Hand over your test to me
   - Leave the room

NB: While inside the classroom, rule 1 and 2 are still valid, even if you've already handed over your answers to me. If you want to discuss your answers with a classmate, or to check an answer on your phone, do that **outside the classroom**.

NB: Make sure to use the restroom, fill our water bottle, grab a coffee, etc... **before** we begin the exam, as you're not allowed to leave the classroom without handing over your answers.

Grading

Problems correctly recognized and solved	Grade
0 - 2	IG
3 - 5	G
6	VG

1 Intro

The local university is migrating to a new system, and they assigned their intern to design the new database schema. Now they've noticed there seems to be some problems with normalization, and they asked us to help them solve them.

As it turns out, the problems aren't due to the intern's lack of knowledge, the intern was in fact much more knowledgeable than it first seemed. A goodbye note was found at their empty desk, where the intern expresses their deep hate for the University, as well as admits to leaving **6 normalization errors** in the design as a *middle-finger* to the institution they so much despise.

It's up to us - as proud alumni of the University - to come to their rescue by normalizing the database.

2 Tables & sample rows

2.1 Students

StudentID	FirstName	LastName	Phones
1	Alice	Andersson	0701234567,0727654321
2	Bob	Berg	0731112222

2.2 Enrollments

StudentID	CourseID	CourseName	Credits	EnrollmentDate	Grade
1	CS101	Intro to Programming	5	2025-01-15	A
1	DB201	Databases	6	2025-01-15	B
2	CS101	Intro to Programming	5	2025-01-15	C

2.3 InstructorCourses

InstructorID	CourseID	InstructorName	InstructorPhone
10	CS101	Dr Smith	0709876543
11	DB201	Dr Karlsson	0708765432

2.4 Courses

CourseID	CourseName	DeptID	DeptName	DeptHead
CS101	Intro to Programming	D1	Computer Science	Prof Adams
DB201	Databases	D1	Computer Science	Prof Adams
MAT100	Calculus I	D2	Mathematics	Prof Brown

2.5 Rooms

RoomID	RoomNumber	BuildingID	BuildingName	BuildingAddress	Capacity
R101	101	B1	Main Hall	Campus Road 1	40
R202	202	B2	Science Center	Science Way 5	35

2.6 Books

ISBN	Title	PublisherName	PublisherAddress
978-1-11111-111-1	Database Systems	TechBooks	123 Tech Ave
978-1-22222-222-2	Intro to Programming	EduPress	45 Learning St

3 SQL code

```
CREATE TABLE Students (
  StudentID INT PRIMARY KEY,
  FirstName VARCHAR(50),
  LastName  VARCHAR(50),
  Phones    VARCHAR(255)
);

CREATE TABLE Enrollments (
  StudentID INT,
  CourseID  VARCHAR(10),
  CourseName VARCHAR(100),
  Credits    TINYINT,
  EnrollmentDate DATE,
  Grade CHAR(1),
  PRIMARY KEY (StudentID, CourseID)
);

CREATE TABLE InstructorCourses (
  InstructorID INT,
  CourseID       VARCHAR(10),
  InstructorName  VARCHAR(100),
  InstructorPhone VARCHAR(15),
  PRIMARY KEY (InstructorID, CourseID)
);
```

```
CREATE TABLE Courses (
  CourseID  VARCHAR(10) PRIMARY KEY,
  CourseName VARCHAR(100),
  DeptID     VARCHAR(10),
  DeptName   VARCHAR(100),
  DeptHead   VARCHAR(100)
);

CREATE TABLE Rooms (
  RoomID          VARCHAR(10) PRIMARY KEY,
  RoomNumber      VARCHAR(10),
  BuildingID      VARCHAR(10),
  BuildingName    VARCHAR(100),
  BuildingAddress VARCHAR(150),
  Capacity INT
);

CREATE TABLE Books (
  ISBN VARCHAR(17) PRIMARY KEY,
  Title VARCHAR(150),
  PublisherName    VARCHAR(100),
  PublisherAddress VARCHAR(150)
);

-- followed by the INSERT statements
--  that match the rows above
```

4 Normalization problems

4.1 1st normalization problem

✓ 1 (2.1) innehåller "phones" två värden. Bryter mot 1NF.
Antingen behåller man bara ett tel.nr eller så skapar man
en separat tabell med kontakt info.

4.2 2nd normalization problem

✓ 1 (2.4). Bryter mot 3NF. Dept.ID, DeptName, DeptHead
bör va i egen tabell då de beror på varandra.

4.3 3rd normalization problem

✓ (2.5) Information om "Building" bör va i egen tabell.
Beror på varandra.
En byggnad kan ha flera rum. Om byggnaden byter
namn måste ändringen ske på flera rader – lätt för fel.

4.4 4th normalization problem

✓ (2.2) credits beror ej på HELA nyckeln, dvs. kvar ej
med Student ID att göra. Bör va i "Courses"-tabellen.

4.5 5th normalization problem

✓ (2.3) Instructor Phone beror ej på HELA nyckeln.
Om instructor ändras står då fel nummer kvar.
Egen tabell för instructor contact info. (ELLER se 4.6)

4.6 6th normalization problem

✓ (2.2) Primärnyckeln bör vara unik. Ex: Enrollment ID

✓ (2.3) Course ID behöver ej va en del av nyckeln.
Tabellen bör heta Instructors. CourseID kan vara
med som referens till courses-tabellen. Då kan
man även behålla InstructorPhone i nuvarande